

1. Kreirati bazu podataka pod imenom **api** sa 3 tabele. Uneti 3 podatka u tabelu *students* pomoću PHP fajla (4. i 5. zadatak).
detects id_detect, ip_address, device_type (*phone, tablet, computer*), user_agent, http_accept, date_time
students id_student, name, city, school, date_time
tokens id_token, token, date_expires, date_time
Na kraju kolokvijuma eksportovati strukturu i podatke iz svih tabela (2 boda).
2. Napisati funkciju *connectToDatabase* koja se povezuje na mysql server (localhost) preko **PDO-a**, vrši odabir baze podataka **api** i vrši podešavanje kodiranja konekcije na utf8mb4. Parametre (5 komada) definisati preko **1 konstante** tipa **niz**. Funkcija treba **STRIKTNO** da primi odgovarajuće parametre i da striktno vrati **objekat PDO klase**. Funkciju smestiti u fajl *functions.php* a parametre u fajl *config.php* (1 bod).
3. Kreirati fajl *token.php* koji će generisati token i uneti vrednost u tabelu *tokens*. Dužina token treba da bude 60 karaktera. Token treba da bude važeći 2 dana (1 bod).
4. Kreirati fajl *students_creator.php* koji poziva funkciju *createStudents*. Funkcija *createStudents* kreira i vraća asocijativni niz *students* sa svim podacima potrebnim za studente (pogledaj tabelu *students*). Podatke generisati po *random* principu. Posebno pokrenuti *random* proces za oba niza. Ove nizove smestiti u *config.php* fajl. Funkcija *createStudents* striktno prima sve potrebne parametre i striktno vraća kreirani niz. Funkciju uneti u fajl *functions.php* (2 boda).

```
$names = ['Marco', 'John', 'Thor', 'Lazarus', 'Edger', 'Smith', 'Timber', 'Rasmus'];  
$citySchool = [  
    ['city' => 'Subotica', 'school' => 'VTS'], ['city' => 'Budapest', 'school' => 'Obuda  
University'], ['city' => 'Belgrade', 'school' => 'University of Belgrade']  
];
```
5. U fajlu *students_creator.php* pozvati pozvati funkciju *insertStudents* koja podatke iz niza *students* upisuje u tabelu *students*. Funkcija *insertStudents* striktno prima parametar. Funkciju uneti u fajl *functions.php* (1 bod).
6. Upotrebom Bootstrap-a kreirati stranicu *index.php* sa 4 forme kao na slici. Podaci sa svih formi se POST metodom šalju na stranicu *request.php*. Svaka forma treba da poseduje **skriveno polje** koje sadrži ime metoda slanja podataka na stranicu *request.php* (3 boda).

POST

Name

City

School

GET

Id

DELETE

Id

PATCH

Id

Name

City

School

7. Stranica *request.php* primenom PHP klase *MobileDetect* detektuje podatke pristupnog uređaja i upisuje ih u tabelu *detects*. Unos podataka uraditi pozivom funkcije *insertDetects* koja striktno prima parametre. Ukoliko metod nije POST, GET, DELETE ili PATCH, preusmeriti korisnika na stranicu *index.php* i ispisati *Method is not allowed* (2 boda).
8. Stanica *request.php* nakon detekcije uređaja i na osnovu vrednosti za metod i na osnovu dobijenih podataka šalje zahtev na stranicu *api.php* primenom **CURL-a**. U svakom zahtevu treba da se šalje *Bearer Token* u *Authorization* zaglavlju. Vrednost za token nabaviti iz tabele *tokens* uz pomoć funkcije *getToken*. Funkcija *getToken* smestiti u fajl *functions.php*. U *CURL* zahtevu dodati opciju *curl_setopt(\$ch, CURLOPT_CUSTOMREQUEST, "\$method")*. Zahtev slati u formatu **api/students**. Za odgovarajuće metode slati odgovarajuće podatke u zahtevu: POST: *name, city i school*, GET: *id*, DELETE: *id*, PATCH: *id, name, city i school* (2 boda).
9. Stranica *api.php* proverava poslati podataka za BEARER token. Ako token nije važeći vratiti odgovor u JSON formatu sa odgovarajućim statusnim kodom i porukom *Token is not valid!* (1 bod).

10. Ukoliko je token važeći i ukoliko je metod POST i ako su ispunjeni svi podaci, stranica *api.php* kreira novi zapis u tabeli *students* (**2 boda**).
11. Ukoliko je token važeći i ukoliko je metod GET i ako je uneta vrednost za id numerički podatak, stranica *api.php* vraća podatke o studentu koji ima taj *id* (**2 boda**).
12. Ukoliko je token važeći i ukoliko je metod DELETE i ako je uneta vrednost za id numerički podatak, stranica *api.php* briše podatke iz tabele *students* za taj *id* (**1 bod**).
13. Ukoliko je token važeći i ukoliko je metod PATCH i ako je uneta vrednost za id numerički podatak i ako postoji vrednost barem za jedno polje (name, city ili school), stranica *api.php* menja podatke za studenta za taj *id* (**2 boda**).
14. Sve metode treba da vrate odgovor u JSON formatu. Odgovor treba da sadrži statusni kod, poruku i podatke (samo u slučaju GET metoda). Ukoliko je uspešno kreiran novi zapis u tabeli *students*, poruka je *Student with id was created*. Ukoliko je metod GET, poruka je *Data for student name*. Ukoliko je metod DELETE, poruka je *Student with id was deleted*. Ukoliko je metod PATCH, poruka je *Student with id was updated* (**1 bod**).
15. Stranica *request.php* ispisuje odgovor dobijen od stranice *api.php*. Odgovor u JSON formatu je potrebno konvertovati u asocijativni niz ispisati odgovarajuće vrednosti (**1 bod**).
16. Podesiti *.htaccess* fajl da *api/students* zahtev bude validan i da se u zahtevu prosleđuje *Authorization* zaglavlje (**1 bod**).

Student: _____